

Tracing dan Visualisasi Algoritma

5027231007 Thio Billy Amansyah

5027241060 Bima Aria Perthama

5027251050 Sean Arthur Tamajaya

5027251096 Afriezal Suryapraba Laiasach

5027251116 Dian Hanna Simanjuntak

I. Tracing Manual

Tracing Manual: Algoritma Dijkstra (Pencarian Rute)

Tujuan: Mencari rute dengan total bobot bahaya paling kecil dari Titik 0 ke Titik 11.

Aturan: Menggunakan **MinHeap** (Antrean Prioritas) untuk selalu mengevaluasi titik dengan akumulasi bobot terkecil. Bobot dihitung menggunakan Mode 6 (Risiko + Kepadatan + Asap).

Persiapan Data (Bobot Mode 6):

- 0 - 3 (Bobot: $9+3+4 = 16$)
- 0 - 4 (Bobot: $9+5+5 = 19$)
- 3 - 11 (Bobot: $3+5+6 = 14$)
- 3 - 16 (Bobot: $1+10+5 = 16$)
- 3 - 23 (Bobot: $3+10+6 = 19$)
- 4 - 9 (Bobot: $5+5+6 = 16$)

Langkah-langkah Eksekusi:

- **Inisialisasi:** `totalRisiko[0] = 0`, titik lain ∞ . `MinHeap = [(Node 0, Total Bobot: 0)]`.
- **Iterasi 1:**
 - **Pop dari Heap:** Node 0 (Bobot 0).
 - **Cek Tetangga:** Node 3 dan Node 4.
 - **Kalkulasi & Update:** * Ke Node 3: Bobot baru $0 + 16 = 16$. Update `totalRisiko[3] = 16`. Masukkan ke Heap.
 - Ke Node 4: Bobot baru $0 + 19 = 19$. Update `totalRisiko[4] = 19`. Masukkan ke Heap.
 - **Kondisi MinHeap Saat Ini:** `[(Node 3, Bobot 16), (Node 4, Bobot 19)]` -> Node 3 berada di urutan pertama karena lebih kecil.
- **Iterasi 2:**
 - **Pop dari Heap:** Node 3 (Bobot 16).
 - **Cek Tetangga:** Node 0, Node 11, Node 16, Node 23.
 - **Kalkulasi & Update:**

- Ke Node 0: Bobot rute putar balik = $16 + 16 = 32$. Ditolak ($32 > 0$).
 - Ke Node 11: Bobot baru $16 + 14 = 30$. Update `totalRisiko[11] = 30`. Masukkan ke Heap.
 - Ke Node 16: Bobot baru $16 + 16 = 32$. Update `totalRisiko[16] = 32`. Masukkan ke Heap.
 - Ke Node 23: Bobot baru $16 + 19 = 35$. Update `totalRisiko[23] = 35`. Masukkan ke Heap.
- **Kondisi MinHeap Saat Ini:** [(Node 4, Bobot 19), (Node 11, Bobot 30), (Node 16, Bobot 32), (Node 23, Bobot 35)] -> Node 4 sekarang di urutan pertama.
- **Iterasi 3:**
 - **Pop dari Heap:** Node 4 (Bobot 19).
 - **Cek Tetangga:** Node 0, Node 9.
 - **Kalkulasi & Update:**
 - Ke Node 9: Bobot baru $19 + 16 = 35$. Update `totalRisiko[9] = 35`. Masukkan ke Heap.
 - **Kondisi MinHeap Saat Ini:** [(Node 11, Bobot 30), (Node 16, Bobot 32), (Node 23, Bobot 35), (Node 9, Bobot 35)]
- **Iterasi 4:**
 - **Pop dari Heap:** Node 11 (Bobot 30).
 - **Cek Target:** Node yang ditarik adalah Target (11). Hentikan pencarian (Break).
 - **Hasil:** Rute aman dikembalikan melalui proses baca mundur *parentNode* ($11 <- 3 <- 0$).

Tracing Manual: Algoritma BFS (Pemetaan Area)

Tujuan: Menyisir/memetakan seluruh ruangan secara merata berdasarkan kedalaman (*Ring*/Jarak Langkah) dari titik awal. Algoritma ini sama sekali tidak memedulikan bobot mode 6. **Aturan:** Menggunakan Antrean Biasa (FIFO *Queue*) dan himpunan penanda ruangan yang sudah dikunjungi (*HashSet visited*).

Langkah-langkah Eksekusi (Mulai dari Node 0):

Inisialisasi: `visited = {0}`. `Queue = [0]`.

Iterasi 1 (Level 0):

- A. **Pop Antrean:** Node 0 dikeluarkan. Cetak ke layar: "0".
- B. **Cek Tetangga (Ring 1):** Node 3 dan 4. Keduanya belum ada di `visited`.
- C. **Tindakan:** Tambahkan 3 dan 4 ke `visited` dan antrean.
- D. **Status:** `visited = {0, 3, 4}`. `Queue = [3, 4]`.

Iterasi 2 (Level 1 - Node Pertama):

- E. **Pop Antrean:** Node 3 dikeluarkan. Cetak ke layar: **"3"**.
- F. **Cek Tetangga (Ring 2 dari jalur 3):** Node 0, 11, 16, 23. Node 0 sudah di **visited**, maka diabaikan.
- G. **Tindakan:** Tambahkan 11, 16, dan 23 ke **visited** dan antrean.
- H. **Status:** **visited** = {0, 3, 4, 11, 16, 23}. **Queue** = [4, 11, 16, 23]. (Node 4 mengantre dari iterasi sebelumnya).

Iterasi 3 (Level 1 - Node Kedua):

- I. **Pop Antrean:** Node 4 dikeluarkan. Cetak ke layar: **"4"**.
- J. **Cek Tetangga (Ring 2 dari jalur 4):** Node 0 dan 9. Node 0 diabaikan.
- K. **Tindakan:** Tambahkan Node 9 ke **visited** dan antrean.
- L. **Status:** **visited** = {0, 3, 4, 11, 16, 23, 9}. **Queue** = [11, 16, 23, 9].

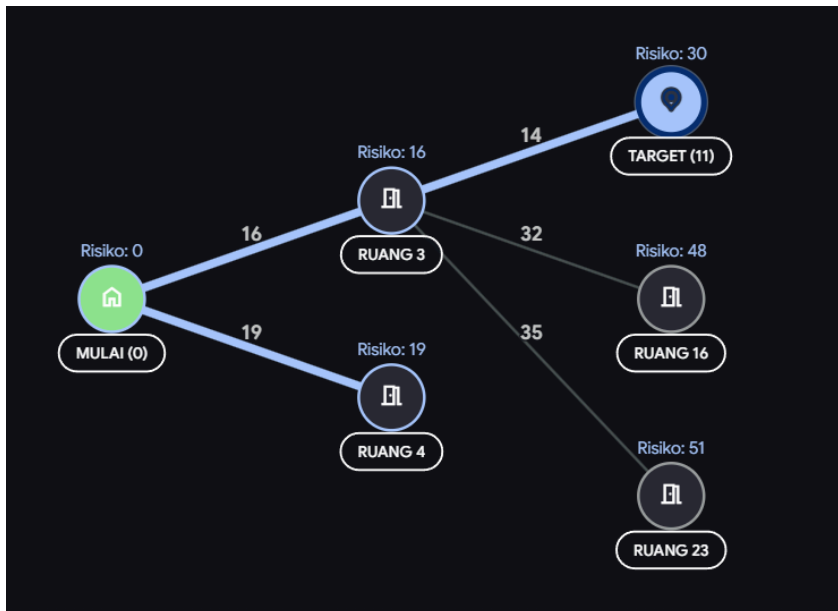
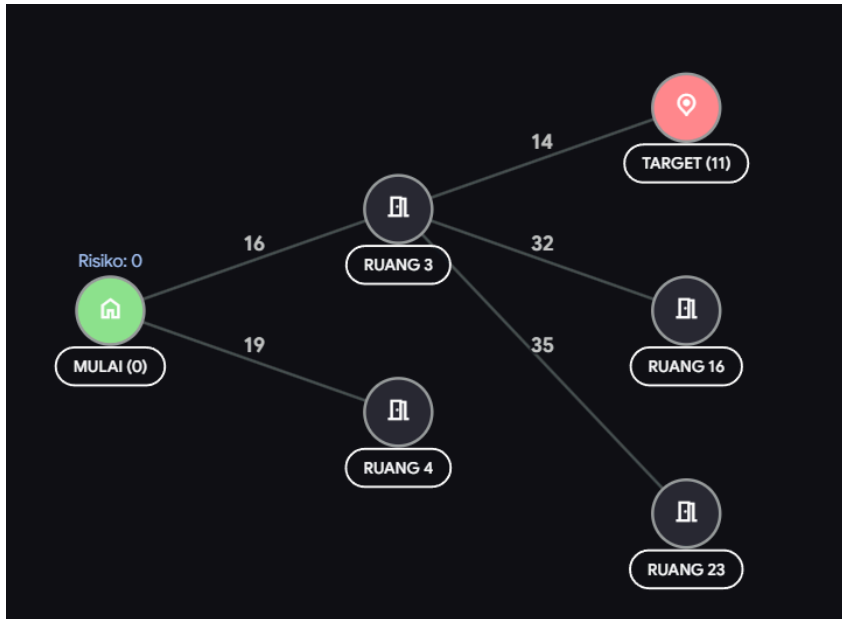
Iterasi 4 (Level 2 - Node Pertama):

- M. **Pop Antrean:** Node 11 dikeluarkan. Cetak ke layar: **"11"**.
- N. **Cek Tetangga (Ring 3 dari jalur 11):** Node 3 dan 10. Node 3 diabaikan karena sudah dikunjungi.
- O. **Tindakan:** Tambahkan Node 10 ke **visited** dan antrean.
- P. **Status:** **visited** = {..., 10}. **Queue** = [16, 23, 9, 10].

Proses ini terus berlanjut hingga Antrean (**Queue**) kosong, dan layar akan mencetak angka berdasarkan kedalaman: 0 lalu 3 4 lalu 11 16 23 9 lalu perluasan dari titik-titik level 2, dan seterusnya.

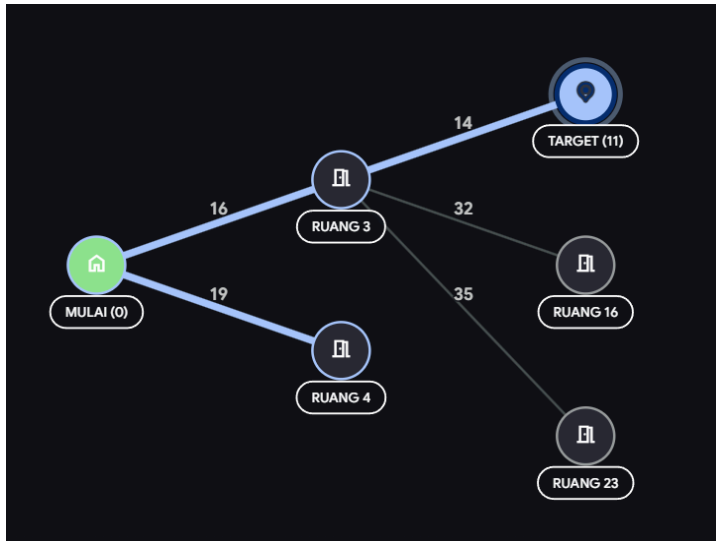
II. Visualisasi

Dijkstra



Dijkstra beroperasi berdasarkan bobot/risiko. Algoritma ini tidak peduli seberapa banyak ruangan yang harus dilewati, asalkan total akumulasi risikonya paling kecil. Algoritma ini menggunakan *Min-Heap* (Antrean Prioritas) untuk selalu mengambil titik dengan total risiko terendah dari titik awal. Jika menemukan jalur baru yang risikonya lebih rendah ke suatu ruangan, ia akan memperbarui rute tersebut.

BFS



Breadth-First Search (BFS) beroperasi berdasarkan tingkat kedalaman/jumlah langkah. Algoritma ini sama sekali mengabaikan bobot (risiko, macet, asap) dan menyebarkan layaknya gelombang air. BFS mengecek semua ruangan tetangga terdekat (Ring 1) terlebih dahulu, kemudian semua tetangga dari tetangga tersebut (Ring 2), dan seterusnya menggunakan struktur data *Queue* (Antrean biasa).

III. Analisis Algoritma

Algoritma Dijkstra (Pendekatan Berbasis Bobot)

Algoritma Dijkstra dirancang secara spesifik untuk memecahkan masalah *Single-Source Shortest Path* pada graf berbobot. Dalam sistem evakuasi ini, "jarak" direpresentasikan sebagai akumulasi bahaya, bukan sekadar jarak fisik.

- Fokus:

Optimasi nilai. Algoritma ini menjamin rute yang dihasilkan adalah rute dengan total risiko paling kecil (gabungan risiko fisik, kepadatan, dan tingkat asap pada Mode 6).

- Mekanisme Eksplorasi:

Bersifat *greedy*. Ia melompat mengevaluasi titik-titik dengan potensi bahaya terkecil terlebih dahulu dengan bantuan *Min-Heap*. Rute yang dilewati bisa saja lebih berliku (melewati banyak ruangan), asalkan koridornya aman.

Algoritma BFS (Pendekatan Berbasis Level/Jarak Hop)

BFS adalah algoritma penelusuran tak berbobot (*unweighted graph traversal*).

- Fokus:

Kedalaman/Jarak topologis. Algoritma ini mengekspansi area layaknya radius ledakan, memetakan ring 1, ring 2, dan seterusnya secara berurutan.

- Mekanisme Eksplorasi:

Adil ke segala arah (*Level-order*). Menggunakan *Queue*, BFS akan mengecek semua ruangan tetangga terdekat secara merata tanpa memedulikan apakah jalur tersebut aman atau dipenuhi asap tebal.

Berikut adalah perbandingan penggunaanya

Parameter Analisis	Algoritma Dijkstra	Algoritma Breadth-First Search (BFS)
Kelebihan Utama	Mampu menghasilkan rute yang dijamin paling aman secara perhitungan matematis. Sangat ideal untuk evakuasi di mana keselamatan (menghindari jalan runtuh/berasap) lebih penting daripada sekadar rute pendek.	Sangat efisien dan cepat dalam pemrosesan data ($O(V+E)$). Ideal untuk melakukan "sapu bersih" atau memetakan area gedung secara utuh untuk melihat konektivitas ruangan.
Kekurangan Utama	Memiliki overhead komputasi yang lebih tinggi akibat operasi priority queue dan relaxation rute.	Buta terhadap bobot/bahaya. BFS dapat mengarahkan korban ke rute dengan tingkat asap level 10 (sangat berbahaya) hanya karena rute tersebut memiliki jumlah pintu/ruangan yang lebih sedikit.
Penggunaan Struktur Data	Memerlukan struktur tambahan (HashMap untuk bobot, Min-Heap untuk antrean).	Sangat sederhana, hanya membutuhkan Queue dasar dan HashSet untuk penanda.

Penerapan kedua algoritma secara bersamaan di dalam struktur program saling melengkapi kebutuhan sebuah simulasi penanganan bencana. Pengembangan struktur graf dinamis berbasis *Adjacency List* memungkinkan peta gedung dimodifikasi (seperti memblokir jalur atau menghapus ruangan yang runtuh total) secara *real-time*.

- **Peran Utama Dijkstra:** Digunakan sebagai inti (*core*) sistem navigasi penyelamatan (*Menu 2*). Ketika pengguna atau korban meminta panduan, Dijkstra melakukan kalkulasi kritis memproses *Min-Heap* untuk memastikan jalur yang diarahkan benar-benar dapat dilalui. Jika ada modifikasi graf (misalnya *deleteEdge* saat simulasi keadaan darurat), Dijkstra akan secara cerdas merutekan ulang mencari jalur memutar.

- **Peran Tambahan BFS:** Berfungsi sebagai modul pemantauan tata ruang atau penyisiran area gedung secara menyeluruh (*Menu 4*). Ini sangat bermanfaat bagi komandan tim SAR yang ingin mengetahui jangkauan persebaran struktur bangunan berawal dari titik komando tertentu secara hierarkis.

Secara keseluruhan, arsitektur yang mengimplementasikan pemrograman berorientasi objek (*Object-Oriented Programming*) ini telah berhasil mendemonstrasikan penyelesaian masalah logika struktur data tingkat lanjut. Integrasi graf dan *tree/heap* khusus di dalamnya membentuk kerangka kerja logis yang sangat solid untuk mengeksekusi perhitungan rute, menunjang kebutuhan pelaporan tugas akhir secara komprehensif.